MCP阶段二 MCP Inspector

原创 已于 2025-11-04 18:44:09 修改 · 710 阅读 · 7 · 0 · CC 4.0 BY-SA版权

文章标签： #人工智能 #AI编程 #mcp #MCP Inspector

介绍

MCP Inspector 是 Model Context Protocol（MCP）官方提供的浏览器端交互式调试工具，用来快速测试、调试任意 MCP 服务器，而无需写代码或配置复杂客户端。一句话：“Postman 之于 HTTP，就是 Inspector 之于 MCP”。

好处：

1. **即开即用**
   一条命令 `npx @modelcontextprotocol/inspector` 自动启动本地 Web UI 与协议代理，零依赖、零安装包残留。

2. **可视化调试**
   左侧树形列出服务器暴露的 Tools / Resources / Prompts；右侧表单填参后即可 **一键调用**，实时查看 JSON-RPC 请求与响应详情 。

3. **多传输协议支持**
   同时支持 stdio、SSE、Streamable HTTP；本地进程或远程地址均可调试。

4. **请求拦截与重放**
   可修改报文再发、保存调用历史、模拟异常响应，方便定位协议兼容性或性能问题。

5. **动态 JSON 表单**
   根据服务器返回的 JSON-Schema 自动生成带默认值、必填校验的输入界面，减少手写 JSON 出错。

6. **安全令牌机制**
   默认生成一次性 Ses

MCP验证工具MCP Inspector使用演示

原创 人工智能 作者：brucexia 时间：2025-10-27 11:13:48

MCP Inspector 是专门为MCP 服务器设计的交互式开发者工具，集成了调试、测试、监控和优化功能。作为MCP 生态的核心调试工具，它通过可视化界面和命令行双模式支持，为开发者提供从协议解析到性能调优的全生命周期支持。其模块化架构设计使其能够适配本地开发环境、云服务集群及混合部署场景，成为连接开发者与MCP 服务器的桥梁。

1. 核心架构解析

1 ）用户界面层

- Web UI 引擎： 基于 React 框架构建的响应式前端系统，支持 Chrome 、 Firefox 等主流浏览器，提供设备自适应布局。包含协议可视化面板、实时数据流图、调试控制台三大核心模块。
- CLI 交互层：提供命令行接口，支持JSON-RPC 协议直连，适用于自动化脚本开发和CI/CD 流水线集成。

2 ）协议解析引擎

- MCP 协议栈实现：完整实现MCP 1.0/2.0 协议规范，包含握手协议解析、消息路由、会话状态跟踪等功能。支持TLS 1.3 加密通信，内置证书管理工具。
- 动态协议适配器：通过插件机制支持WebSocket 、gRPC 、HTTP/2 等传输协议，开发者可自定义协议解析规则。

3 ）传输适配层

- 多通道管理器：集成TCP 、UDP 、SSE 三种传输通道，支持QoS （Quality of Service ，服务质量）分级设置（如低延迟通道用于调试指令，高吞吐通道用于日志传输）。
- 网络模拟器：提供带宽限制、延迟注入、丢包模拟等网络异常测试功能，帮助验证服务器的容错能力。

4 ）插件扩展模块

- 核心插件集：包含JSON Schema 验证器、OpenTelemetry 集成器、Prometheus 指标采集器等12 个官方插件。
- 开发者SDK ：提供TypeScript/Python 绑定接口，支持自定义插件开发，包含调试钩子（Hook ）、事件订阅等扩展点。

5 ）日志监控系统

- 多级日志架构：实现DEBUG 、INFO 、WARN 、ERROR 四级日志体系，支持结构化日志输出（JSON 格式）和文本日志混合模式。
- 实时监控仪表盘：集成Grafana 模板，展示TPS 、内存占用、请求延迟等20+ 关键指标。

6 ）协同调试模块

- 多用户会话共享：支持基于WebSocket 的实时协作调试，允许多个开发者同时操作同一调试会话，具备操作权限分级控制。
- 版本对比工具：可并行加载多个MCP 服务器版本的调试数据，自动生成差异分析报告。

2. 核心功能详解

1 ）交互式调试环境

- 断点管理：支持在协议握手阶段、消息处理函数、状态转换节点设置断点，提供条件断点（如特定请求ID 触发）和计数断点（执行 *N* 次后触发）。
- 变量观察窗：实时显示服务器上下文变量，包括连接状态机、会话令牌、缓存命中率等关键参数。
- 热代码替换：在调试过程中动态更新服务器端处理逻辑，无须重启服务即可验证修复方案。

2 ）可视化数据追踪

- 消息流向拓扑图：动态生成客户端－服务器交互的时序图，支持Zoom/Filter 操作，高亮异常消息路径。
- 性能火焰图：基于采样数据生成CPU/ 内存消耗分布图，精准定位热点函数。
- 协议合规性矩阵：将实际通信数据与MCP 规范对比，自动生成符合度报告（Conformance Report ），如必填字段缺失、非法状态转换等。

3 ）自动化测试框架

- 测试用例生成器： 基于协议规范自动生成边界值测试用例，如最大消息长度、最小间隔时间。
- 混沌工程模块：集成Chaos Monkey 原理，随机注入网络分区、服务降级等故障场景。
- 回归测试套件：支持测试用例版本管理，每次提交代码后自动运行历史测试集。

4 ）协同调试工作流

- 会话录制与回放：完整记录调试过程（包括网络流量、断点触发顺序），支持导出为YAML 测试脚本。
- 问题追踪集成：与Jira/GitHub Issues 深度集成，可一键创建Issue 并附加调试日志。
- 远程协助模式：通过共享调试令牌，允许外部专家接入当前调试会话，权限可配置为只读或编辑。

3. Inspector 的组成模块与具体使用

Inspector 组成模块如图11-1 所示。其中最重要的模块是Tools 、Resources 与Prompts 。

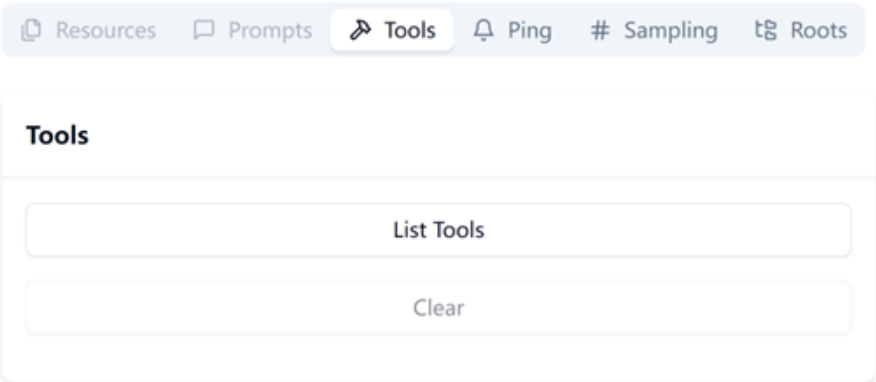

图11-1 Inspector 的组成模块

1 ）Tools

MCP Inspector 中的Tools 是一种功能模块，它赋予服务器公开可执行函数的能力。这些函数可被客户端调用，并被大型语言模型用于执行各类操作。Tools 的独特之处在于，它不仅使LLM 能够从外部获取所需信息，还支持执行写入或操作任务，从而为LLM 赋予真正意义上的行动力。在模型控制方面，Tools 直接将可执行函数暴露给LLM ，使得模型能够主动发起调用，灵活地完成相应任务。

Tools （工具）本质上是一种依托浏览器环境实现的函数调用机制，其核心是通过标准化的 JSON-RPC 协议实现工具调用与交互。以下是对其核心机制的说明：

其一为工具列表注册函数，其请求命令示例如下：

```
{
    "method": "tools/list",
    "params": {},
    "jsonrpc": "2.0",
    "id": 1
}
```

……

其二为工具描述函数，其请求命令示例如下：

```
{
    "method": "tools/call",
    "params": {
        "name": "fetch_weather",
        "arguments": {
```

"url": "https://www.weather.com.cn"

},

"_meta": {

"progressToken": 0

}

},

"jsonrpc": "2.0",

"id": 2

}

从上面示例代码 中可以看到 Tools 提供了两个主要的注册函数。 其中一个函数允许注册一个工具，该工具接受给定的参数，这些参数必须是一个包含与其模式相关联的命名属性的对象。当客户端调用这个工具时，函数会使用经过解析和验证的参数来运行。

另一个函数则在此基础上增加描述字段，用于对工具进行更详细的说明。在具体实现里，Tools 将函数调用进行了标准化处理，包含了工具名称和描述。这些信息不仅有助于LLM 理解工具的用途，还可能在其他场景发挥作用。同时，它定义了强类型的参数，用于对输入参数进行验证，确保数据的准确性和安全性。此外，它还规定了标准化的返回格式来保证返回数据的结构一致性。

2 ）Resources

Resources 用于表示服务器希望向客户端提供的各类只读数据。这些数据形式多样，涵盖文件内容、数据库记录、图片、日志等多种类型。

在应用控制上，Resources 由客户端或应用程序进行管理，其主要作用是为LLM 提供必要的上下文内容，助力LLM 更好地理解和处理信息。

Resources 是对数据访问的一种抽象。Resources 提供了两种实现方式，如下所示：

1.[协议]://[主机]/[路径]

2.[protocol]://[host]/[path]

第一种是静态资源，比如定义一个“API 文档”资源，通过指定路径和处理函数，返回包含文档内容的标准化数据格式。第二种是动态资源模板，例如“用户数据”资源，使用模板化的URI （[host]/[path] ），根据不同的参数动态生成数据，同样遵循标准化的数据返回格式。Resourcess 统一了URI 寻址方式，不仅支持常规的URI 格式，还提供了模板支持，方便根据不同的参数生成对应的资源地址；它还明确了内容类型声明，让客户端能够准确识别资源的类型，并且规定了标准化的数据返回格式，确保不同资源返回的数据结构一致，便于客户端处理。

3 ）Prompts

Prompts 是由服务器精心定义的可重用模板。用户能够从中进行选择，以此引导或规范与LLM 的交互流程。例如，Git MCP server 可以提供一个“生成提交信息”的提示词模板，用户借助该模板，能够创建出标准化的提交消息。在用户控制层面，Prompt 通常由用户根据自身需求自行选择使用。

Prompts 主要用于统一提示词模板的定义和使用，示例代码如下：

```
const codeReviewTool = {

  name: 'code_review',

  description: 'Automatically review code and provide suggestions',

  parameters: {

    type: 'object',

    properties: {

      filePath: { type: 'string', description: 'Path to the file being reviewed' },

      code: { type: 'string', description: 'Code content to review' },

      context: { type: 'string', description: 'Additional context for review' }

    },
```

```
    required: ['filePath', 'code']
  },
  handler: async ({ filePath, code, context }) => {
    const prompt = `
      请对以下代码进行审查：
      文件路径 : ${filePath}
      代码内容 :
      ${code}

      ${context ? ` 上下文 : ${context}` : ''}

      请指出潜在问题、改进建议和最佳实践。
    `;

    const response = await openrouterClient.generate(prompt);
    return { review: response };
  }
};
```

从上面示例代码可知，Prompts 定义了一个名为code-review 的提示词模板，它包含参数化提示词定义，通过Zod 模式定义了提示词所需的参数类型，如代码、语言和可选的重点关注内容；同时保证了类型安全，在参数定义和数据处理过程中进行类型检查；还规定了标准化的消息格式，以特定的结构组织提示词内容，便于客户端理解和使用。

此外，MCP 还提供了一个可选的高级功能— 能力协商。能力协商类似于双方见面时的自我介绍，用于协商版本升级等握手过程。在初始化请求中，客户端会声明自己支持的能力，如协议版本、客户端信息以及所支持的功能（如是否支持被反向调用LLM ，是否支持文件系统访问等）。服务器收到请求后，会回复自己所支持的能力，如是否支持工具、资源和提示词等功能。通过这种协商机制，客户端和服务端可以根据对方支持的能力来决定如何进行交互。例如，当某个工具升级或废弃时，可以通过能力协商告知客户端。不过，这种能力协商机制需要客户端和服务端一对一单独开发，无法实现多个服务的通用。

MCP Inspector 中的Tools 是一种功能模块，它赋予在浏览器公开可执行函数的能力。这些函数可被调用，并由大型语言模型（LLM ）用于执行各类操作。Tool 的独特之处在于，它不仅使LLM 能够从外部获取所需信息，还支持执行写入或操作任务，从而为LLM 赋予了真正意义上的行动力。在模型控制方面，Tool 直接将可执行函数暴露给LLM ，使得模型能够主动发起调用，灵活地完成相应任务。

Tools （工具）本质上是一种依托浏览器环境实现的函数调用机制，其核心是通过标准化的 JSON-RPC 协议实现工具调用与交互。以下是对其核心机制的说明：

其一为工具列表注册函数，其请求命令示例如下：

```
{
    "method": "tools/list",
    "params": {},
    "jsonrpc": "2.0",
    "id": 1
}
```

……

其二为工具描述函数，其请求命令示例如下：

```
{
    "method": "tools/call",
    "params": {
        "name": "fetch_weather",
        "arguments": {
            "url": "https://www.weather.com.cn"
        },
        "_meta": {
            "progressToken": 0
        }
    },
    "jsonrpc": "2.0",
    "id": 2
}
```

可以看到，Tools 提供了两个主要的浏览器调用命令。 其中一个函数允许注册一个工具，注册的工具可接受特定参数。这些参数必须是包含与工具模式关联的命名属性的对象。当客户端调用工具时，系统会先对参数进行解析和验证，再运行工具函数。

另一个函数则在此基础上增加了描述字段，用于对工具进行更详细的说明。在具体实现里，Tools 将函数调用进行了标准化处理。它包含了工具名称和描述，这些信息不仅有助于LLM 理解工具的用途，还可能在其他场景发挥作用；同时，它定义了强类型参数注册，用于对输入参数进行验证，从而确保数据的准确性和安全性；此外，它还规定了标准化的返回格式来保证返回数据的结构一致性。

==========

本文节选自《AI Agent智能体与MCP开发实践：基于Qwen3大模型》，获得出版社和作者授权发布。